Challenges

PWAs require application architecture, including event management, state and caching. Further, team members at the front-end, back-end and those working on the full stack should be trained in the new structures and tool chain. PWA-building expertise needs to go beyond JQuery and HTML.

Open source technologies such as Angular or React are just libraries for the user interface (UI). They provide low-level tools for managing events and state. Progressive generators solve only the most straightforward part of building a PWA, such as generating the manifest or essential service worker. Besides, they don't come with out-of-the-box integration for significant e-commerce and marketing platforms. Nor can they optimise battle-hardened components for speed and conversion.

Making PWAs run forever

The Indian PWA development industry is quite mature compared to various other countries. It caters not only to a thriving and challenging domestic market but also to the global market. As the needs of the market evolve, the features of PWAs also change from time to time. PWAs can become the future of Web development provided they keep pace with rapid changes in technology.

If a company can put together the best PWAs money can buy, super-efficient Web app programmers and decades of Web app-making know-how, then, theoretically, a Web device might run forever. END

References

[1] *https:// www.yourteamindia.com*
[2] A blog on PWAs by *spec-india.com*
[3] *https://www.dizzain.com> insights*
[4] *https://www.webdesignexpress.com*
[5] vilmate.com
[6] 'The pros and cons of PWAs' by Uzair Khan
[7] '10 challenges for building a PWA' by Peter Mclachlan
[8] Several other Web pages were referred to while writing this article.

By: Vinayak Ramachandra Adkoli

The author is a B.E. in industrial production. He has 10 years of experience as a lecturer in the mechanical department of three different polytechnics. He is also a freelance writer and cartoonist.

Continued from page 48...

```
Session {
  cookie:
   { path: '/',
     _expires: null,
     originalMaxAge: null,
     httpOnly: true },
  visits: 3,
  email: 'testusername' }
```

variable 'visits' is updated to three when the user reloads the page

Figure 7: Updated console output for *console.log (req.session)* with an increasing number of visits

You need to log in before you can see this page.

Login

Figure 8: Page shows that the user has not logged in properly

```
Session {
  cookie:
   { path: '/',
     _expires: null,
     originalMaxAge: null,
     httpOnly: true },
  visits: 1 }
```

Figure 9: Console output for *console.log (req.session)* when user is not logged in properly

As you can see in Figure 7, the visits variable has become 3, and that's what it displays on the page as seen in Figure 6b.

Let us now test what the website does when we try visiting *http://localhost:3000/admin* without being logged in (Figure 8).

Figure 9 shows what we have on the console now.

As we can see in Figure 9, there is no email variable in the cookie being sent, and thus no user is logged in this time; so the website prompts the user to the login page with the link.

In this tutorial, we have successfully managed to validate user sessions with a simple login page by using Express' simple and effective implementation of session management. Hopefully, you can follow along and recreate the same output with your own forms and validations. END

References

[1] *https://www.npmjs.com/package/express-session*
[2] *https://dzone.com/articles/securing-nodejs-managing-sessions-in-expressjs*

By: Kshitij Nair

The author is an open source enthusiast. His key interests are Web and mobile development.

Acknowledgements

The author is grateful to Dr Sibi Chakkaravarthy Sethuraman and Dr Anupama Namburu at the School of Computer Science and Engineering, VIT-AP University and to Dr Hari Seetha, dean, School of Computer Science and Engineering, VIT-AP University.